Skúška inteligencie

2. zadanie – deadline: 24. 4. 2026

Alan Turing bol jeden z priekopníkov počítačovej vedy a umelej inteligencie. Prvé výsledky jeho úvahy o počítačoch publikoval pred 90 rokmi (*On Computable Numbers, with an Application to the Entscheidungsproblem*). Ďalší dôležitý míľnik vo vývoji umelej inteligencie predstavuje jeho článok z roku 1950 (*Computing Machinery and Intelligence*), v ktorom popísal spôsob, ako určiť, či stroj je inteligentný, dnes známy ako Turingov test.

Pri príležitosti tohto výročia sa jeden výskumný tím rozhodol systematicky analyzovať kvalitu rôznych konverzačných AI systémov pomocou kontrolovaných experimentov. Každý experiment pozostáva z definovaného protokolu hodnotenia, ktorý špecifikuje jednotlivé kroky analýzy a odborníkov potrebných na ich realizáciu. Experimenty vykonáva výskumné laboratórium, ktoré má obmedzený počet expertov rôznych špecializácií a musí efektívne rozhodovať, ktoré experimenty bude realizovať a v akom poradí. Vašou úlohou je pomôcť im namodelovať problém a efektívne rozdeliť výskumné kapacity na jednotlivé úlohy.

Cieľom zadania je navrhnúť objektovo orientovaný model tejto situácie v jazyku Python. Vytvoríte niekoľko navzájom prepojených tried reprezentujúcich: AI systémy, ktoré budú predmetom hodnotenia (trieda `AISystem`), expertov rôznych špecializácií, ktorí sa podieľajú na hodnotení (všeobecná trieda `Expert` a podtriedy `LinguistExpert`, `MLResearcher` a `Statistician`), experimentálne protokoly definujúce jednotlivé kroky hodnotenia (triedy `Step` a `EvaluationProtocol`), samotné experimenty, ktoré simulujú priebeh testovania (trieda `Experiment`), výskumné laboratórium, ktoré experimenty organizuje (trieda `ResearchLab`), a rôzne stratégie výberu experimentov (všeobecná `ExperimentStrategy` a konkrétne `ChronologicalStrategy`, `ComplexProtocolStrategy` a `EfficientStrategy`).

Triedy budú spolupracovať pri simulácii procesu hodnotenia AI systémov počas viacerých dní. Pri implementácii sa zameriate najmä na objektovo orientované programovanie, dedenie, prácu s kolekciami objektov a návrh jednoduchých rozhodovacích algoritmov.

Poznámka: Triedy viete implementovať zväčša nezávisle, avšak pre implementáciu musíte pochopiť ich vzájomné prepojenia. Popis štruktúry tried v tomto súbore je minimálny, teda uvedené premenné a metódy musia existovať v triedach, môžete ale pridať ďalšie podľa potreby. Nemeňte názvy definovaných atribútov, ani parametre zadaných metód. Pri odovzdaní riešenia nahrajte všetky vypracované súbory (`ai_system.py`, `evaluation.py`, `experiment.py`, `expert.py`, `lab.py`) na Google Drive.

Pre lepšiu čitateľnosť zadania je kľúčové slovo `self` vynechané pri členských premenných a zo zoznamu parametrov metód.

Trieda `AISystem` – 0,5 bodu

Trieda `AISystem` reprezentuje systém umelej inteligencie, ktorý bude podrobený experimentálnemu hodnoteniu. Každý systém má definovanú špecializáciu, teda úlohu, na ktorú je určený, technologický framework a kvalitu modelu, ktorá ovplyvňuje výsledky simulovaných experimentov. Trieda obsahuje nasledovné členské premenné:

- `name` – `string` – názov AI systému,

- `specialization` – `string` – doména, na ktorú je AI systém zameraný,
- `framework` – `string` – technologický framework použitý pri implementácii systému,
- `quality` – `float` – hodnota z intervalu ⟨0, 1⟩ reprezentujúca kvalitu modelu.

V triede implementujte konštruktor (0,1 bodu), ktorý nastaví hodnoty členských premenných podľa parametrov. V triede okrem toho implementujte nasledovné metódy:

- **(0,2b) simulate_dialogue()** – metóda simuluje jeden dialóg so systémom a vráti slovník obsahujúci metriky kvality odpovedí. Hodnoty generujte pomocou náhodných čísel tak, aby záviseli od kvality systému (členská premenná `quality`). Vyššia kvalita vedie k vyššej koherentnosti a konzistentnosti, menšiemu počtu gramatických chýb a bohatšej slovnej zásobe. Návratová hodnota – slovník – obsahuje nasledovné položky:
  - *coherence* – hodnota typu `float` z intervalu ⟨0, 1⟩ reprezentujúca koherentnosť odpovedí. Hodnotu vygenerujte z normálneho rozdelenia s priemerom `quality` a odchýlkou 0,1.
  - *consistency* – hodnota typu `float` z intervalu ⟨0, 1⟩ reprezentujúca konzistentnosť odpovedí. Hodnotu vygenerujte z normálneho rozdelenia s priemerom `quality` a odchýlkou 0,07.
  - *grammar_errors* – celé číslo reprezentujúce počet gramatických chýb. Hodnotu vygenerujte z normálneho rozdelenia s priemerom `10 * (1 - quality)` a odchýlkou 1.
  - *vocabulary_richness* – hodnota typu `float` z intervalu ⟨0, 1⟩ reprezentujúca bohatosť slovnej zásoby. Hodnotu vygenerujte z normálneho rozdelenia s priemerom `0,5 + 0,5 * quality` a odchýlkou 0,1.
- **(0,2b) get_dialogue_metrics(n_dialogues)** – metóda vykoná simuláciu viacerých dialógov a vráti priemerné hodnoty metrík kvality. Parameter `n_dialogues` udáva počet simulovaných dialógov (celé číslo, predvolená hodnota je 100). Návratová hodnota je slovník s rovnakými kľúčmi ako metóda `simulate_dialogue()`. Výsledné hodnoty vypočítajte ako priemer z nasimulovaných dialógov, priemerný počet gramatických chýb nech je celé číslo.

Trieda `Expert` – 1 bod

Trieda `Expert` reprezentuje všeobecného odborníka, ktorý sa podieľa na hodnotení AI systémov. Každý expert má definovanú oblasť špecializácie, akademickú pozíciu a mieru spoľahlivosti, ktorá ovplyvňuje kvalitu jeho hodnotení. Trieda obsahuje nasledovné členské premenné:
- `name` – `string` – meno experta,
- `specialization` – `string` – oblasť odbornosti experta,
- `seniority` – `string` – akademická pozícia experta, povolené hodnoty sú *assistant*, *associate* a *professor*,
- `reliability` – `float` – hodnota z intervalu ⟨0, 1⟩ vyjadrujúca spoľahlivosť experta.

V triede implementujte konštruktor (0,5b), ktorý nastaví hodnoty členských premenných podľa vstupných parametrov. Pri vytváraní objektu skontrolujte správnosť vstupných údajov a v prípade chyby vygenerujte príslušnú výnimku podľa nasledovných pravidiel:

- ak hodnota parametra `seniority` nepatrí medzi povolené hodnoty, vyvolajte výnimku typu `ValueError` so správou *Unknown seniority level: XY*, kde *XY* je zadaná hodnota.

- ak hodnota parametra `reliability` nie je typu `float`, vyvolajte výnimku typu `TypeError` so správou *Reliability must be float, not XY*, kde *XY* je typ zadanej hodnoty.
- ak hodnota parametra `reliability` nie je z intervalu ⟨0, 1⟩, vyvolajte výnimku typu `ValueError` so správou *Reliability must be between 0 and 1. Got XY*, kde *XY* je zadaná hodnota.
- ak expert na pozícii *assistant* má spoľahlivosť väčšiu ako 0,6, vyvolajte výnimku typu `ValueError` so správou: *An assistant's reliability cannot be bigger than 0.6*
- ak expert na pozícii *associate* má spoľahlivosť menšiu ako 0,5 alebo väčšiu ako 0,8, vyvolajte výnimku typu `ValueError` s príslušnou z nasledovných správ *An associate's reliability cannot be smaller than 0.5* alebo *An associate's reliability cannot be bigger than 0.8*
- ak expert na pozícii *professor* má spoľahlivosť menšiu ako 0,7, vyvolajte výnimku typu `ValueError` so správou *A professor's reliability cannot be smaller than 0.7*

Ak je niektorá hodnota neplatná, objekt triedy sa nesmie vytvoriť. Okrem konštruktora implementujte nasledovné metódy:

- **(0,3b) estimate_step_duration(base_hours)** – metóda odhaduje počet hodín potrebných na vykonanie kroku experimentu na základe seniority experta. Parameter `base_hours` je celé číslo reprezentujúce základný odhad trvania kroku. Metóda vracia celé číslo vypočítané nasledovne:
  - expert na pozícii *assistant* potrebuje o 20 % viac času,
  - expert na pozícii *associate* potrebuje presne odhadovaný čas,
  - expert na pozícii *professor* potrebuje o 20 % menej času.

  Výsledný počet hodín vždy zaokrúhlite smerom nahor (počítate začaté hodiny).
- **(0,2b) compute_reliability_weight(system_specialization)** – metóda upraví spoľahlivosť experta v závislosti od toho, či sa špecializácia experta zhoduje so špecializáciou hodnoteného AI systému (parameter `system_specialization`, typ `string`). Ak sa špecializácie zhodujú, výsledná spoľahlivosť je zvýšená o 10 %, maximálne však na hodnotu 1,0. V opačnom prípade sa spoľahlivosť nemení. Metóda vracia výslednú hodnotu typu `float`.

Trieda `LinguistExpert` – 0,5 bodu

Trieda `LinguistExpert` reprezentuje lingvistického odborníka, ktorý hodnotí kvalitu jazykového prejavu AI systému. Tento typ experta sa zameriava najmä na koherentnosť textu, bohatosť slovnej zásoby a počet gramatických chýb. Trieda `LinguistExpert` má byť podtriedou triedy `Expert` a dediť všetky jej atribúty aj metódy. Implementujte konštruktor s rovnakými parametrami ako v triede `Expert` a zabezpečte, aby sa pri vytváraní objektu vykonali všetky kontroly definované v rodičovskej triede. Okrem toho implementujte nasledovné metódy:

- **(0,4b) evaluate_dialogue(dialogue_metrics)** – metóda vyhodnotí kvalitu dialógu AI systému na základe jazykových metrík. Parameter `dialogue_metrics` je slovník obsahujúci nasledovné položky:
  - *coherence* – hodnota typu `float` z intervalu ⟨0, 1⟩,
  - *vocabulary_richness* – hodnota typu `float` z intervalu ⟨0, 1⟩,
  - *grammar_errors* – celé číslo reprezentujúce počet gramatických chýb.

  Výsledné hodnotenie vypočítajte ako vážený priemer metrík, pričom váhy závisia od seniority experta:

seniorita	váha koherentnosti	váha slovnej zásoby	váha gramatiky
assistant	0.7	0.2	0.1
associate	0.6	0.3	0.1
professor	0.5	0.3	0.2

Počet gramatických chýb transformujte na hodnotu kvality pomocou výrazu `1 / (1 + grammar_errors)`.
Výsledkom metódy je číslo typu `float` z intervalu ⟨0, 1⟩ reprezentujúce jazykovú kvalitu dialógu.
Ak slovník `dialogue_metrics` neobsahuje niektorý z požadovaných kľúčov, vyvolajte výnimku typu `KeyError` so správou *Did not find XY in metrics*, kde *XY* je chýbajúci kľúč.

- **(0,1b) estimate_step_duration(base_hours)** – metóda prepisuje implementáciu z rodičovskej triedy `Expert`, pričom expert na pozícii *assistant* potrebuje o 30 % viac času a profesor o 25 % menej času. Výsledný počet hodín vždy zaokrúhlite smerom nahor.

Trieda `MLResearcher` – 1 bod

Trieda `MLResearcher` reprezentuje výskumníka v oblasti strojového učenia, ktorý analyzuje stabilitu a spoľahlivosť AI systému pomocou opakovaných meraní kvality jeho výstupov. Tento expert sa zameriava najmä na konzistentnosť modelu a citlivosť jeho výkonu na rôzne vstupy. Trieda `MLResearcher` má byť podtriedou triedy `Expert` a dediť všetky jej atribúty a metódy. Okrem atribútov definovaných v triede `Expert` má trieda `MLResearcher` jednu dodatočnú členskú premennú:
- `known_frameworks` – `list` – zoznam frameworkov, s ktorými má expert skúsenosti.

Implementujte konštruktor (0,1b) s rovnakými parametrami ako v triede `Expert`, rozšírený o parameter `frameworks`, ktorý predstavuje zoznam frameworkov. Hodnotu tohto parametra uložte do členskej premennej `known_frameworks`.

V triede implementujte nasledovné metódy:

- **(0,1b) knows_framework(framework)** – metóda zistí, či expert pozná zadaný framework (parameter `framework` typu `string`) a vráti hodnotu `True` alebo `False`.
- **(0,8b) validate_model(ai_system, hours)** – metóda vykoná experimentálne overenie stability modelu. Parameter `ai_system` je objekt triedy `AISystem` a parameter `hours` predstavuje počet hodín vyhradených na experiment (celé číslo). Skutočný počet hodín experimentu získajte pomocou metódy `estimate_step_duration()` zdedenej z triedy `Expert`. Počas experimentu vykonajte opakované simulácie dialógu. Počet simulácií odvoďte zo skutočného počtu hodín experimentu, ak viete že expert stihne vykonať dva experimenty za jednu hodinu. Z každej simulácie zachovajte hodnoty metrík coherence a consistency, z nich následne vypočítajte rozptyl (variance) ako priemerný štvorcový rozdiel od priemeru. Ak expert pozná framework použitého AI systému, výsledný rozptyl vynásobte koeficientom 0,8. Celkovú mieru variability určte ako aritmetický priemer rozptylov koherentnosti a konzistentnosti.
  Metóda vracia slovník obsahujúci položky:
  - *coherences* – zoznam hodnôt koherentnosti zo všetkých simulácií,

- *consistencies* – zoznam hodnôt konzistentnosti zo všetkých simulácií,
- *variability* – hodnota typu `float` reprezentujúca výslednú mieru variability modelu.

Trieda `Statistician` – 1 bod

Trieda `Statistician` reprezentuje odborníka, ktorý agreguje čiastkové výsledky hodnotenia AI systému do jednej výslednej metriky. Tento expert kombinuje hodnotenie jazykovej kvality a technickej stability modelu a zohľadňuje pritom spoľahlivosť jednotlivých expertov. Trieda `Statistician` má byť podtriedou triedy `Expert` a dediť všetky jej atribúty aj metódy. Implementujte konštruktor s rovnakými parametrami a funkcionalitou ako v rodičovskej triede.

V triede implementujte nasledovnú metódu:

- **(1b) aggregate_scores(metric_dict)** – metóda vypočíta celkové hodnotenie AI systému na základe výsledkov jednotlivých expertov. Parameter `metric_dict` je slovník v nasledovnom formáte:

  {
  "*quality*": (ai_system, quality_score),
  "*linguist*": (linguist_expert, linguist_score),
  "*MLexpert*": (ml_expert, ml_results)
  }

  kde `ai_system` je objekt triedy `AISystem`, `quality_score` je hodnota kvality systému, `linguist_score` je výsledok metódy `evaluate_dialogue()` triedy `LinguistExpert`, `ml_results` je slovník vrátený metódou `validate_model()` triedy `MLResearcher`, obsahujúci zoznamy hodnôt `coherences`, `consistencies` a hodnotu `variability`.

  Pri výpočte postupujte nasledovne:

  1. Z dát poskytnutých expertom typu `MLResearcher` vypočítajte priemernú hodnotu koherentnosti a konzistentnosti ako aritmetický priemer hodnôt v zoznamoch `coherences` a `consistencies`.
  2. Vypočítajte mieru robustnosti modelu podľa vzťahu:
     robustnosť = priemer(koherentnosť, konzistentnosť) × (1 / (1 + variability))
  3. Určte váhy jednotlivých expertov pomocou metódy `compute_reliability_weight()`. Váha závisí od toho, či sa špecializácia experta zhoduje so špecializáciou AI systému.
  4. Kombinujte výsledky lingvistického experta a ML experta pomocou váženého priemeru:
     (váha_ML × robustnosť + váha_lingvista × linguist_score) / (váha_ML + váha_lingvista)
  5. Výsledné hodnotenie systému vypočítajte ako kombináciu pôvodnej kvality modelu a expertného hodnotenia:
     váha_statistika × (0.6 × quality_score + 0.4 × expertné_hodnotenie)
     kde váha_statistika je hodnota získaná metódou `compute_reliability_weight()` pre objekt `Statistician`.

  Metóda vracia výsledné hodnotenie ako číslo typu `float`.

Trieda `Step` – 0,2 bodu

Trieda `Step` reprezentuje jeden krok vyhodnotenia systému. Každý krok má slovný popis, odhadovaný počet hodín potrebných na jeho vykonanie a zoznam typov expertov, ktorí sa môžu na danom kroku podieľať. Trieda obsahuje nasledovné členské premenné:
- `step_desc` – `string` – stručný popis kroku experimentu,
- `duration` – `int` – počet hodín potrebných na vykonanie kroku,
- `experts` – `list` – zoznam tried expertov, ktoré sa môžu podieľať na danom kroku (hodnoty z `LinguistExpert`, `MLResearcher`, `Statistician`).

V triede implementujte konštruktor (0,1b), ktorý nastaví hodnoty členských premenných podľa vstupných parametrov. Okrem konštruktora implementujte metódu `can_contribute(expert_type)` (0,1b), ktorá zistí, či sa expert daného typu môže podieľať na kroku experimentu. Parameter `expert_type` predstavuje triedu experta (napr. `LinguistExpert`).

Trieda `EvaluationProtocol` – 0,8 bodu

Trieda `EvaluationProtocol` reprezentuje experimentálny protokol, ktorý definuje postup hodnotenia AI systému. Protokol pozostáva z viacerých krokov experimentu, pričom každý krok určuje časovú náročnosť a typ expertov potrebných na jeho realizáciu. Trieda obsahuje nasledovné členské premenné:
- `protocol_name` – `string` – názov protokolu,
- `steps` – `list` – zoznam krokov experimentu (objekty triedy `Step`),
- `required_experts` – `dict` – slovník, ktorý ku každému kroku priraďuje zoznam typov expertov potrebných na jeho vykonanie.

Implementujte konštruktor (0,1b), ktorý nastaví názov protokolu, inicializuje prázdny zoznam krokov a prázdny slovník požadovaných expertov. V triede okrem toho implementujte nasledovné metódy:

- **(0,3b) add_step(step_info)** – metóda pridá nový krok do protokolu. Parameter `step_info` je slovník obsahujúci informácie o kroku v nasledovnom formáte:
  {
  "*desc*": slovný popis kroku,
  "*duration*": počet hodín potrebných na vykonanie kroku,
  "*requirements*": zoznam tried expertov, ktoré sa môžu podieľať na kroku
  }.
  Metóda vytvorí nový objekt triedy `Step` a pridá ho do zoznamu `steps`. Zároveň aktualizuje slovník `required_experts` tak, aby pre každý krok (kľúče) obsahoval zoznam požadovaných typov expertov (hodnoty). Metóda nevracia žiadnu hodnotu.
  Ak hodnota *duration* nie je celé číslo alebo je menšia ako 1, vyvolajte výnimku typu `ValueError` so správou *Step duration must be an integer and at least 1*
- **(0,2b) get_all_experts()** – metóda vráti zoznam všetkých typov expertov, ktorí sú potrební na realizáciu protokolu. Každý typ experta sa má v zozname nachádzať najviac raz, aj keď sa vyžaduje vo viacerých krokoch.
- **(0,1b) get_total_duration()** – metóda vráti celkový počet hodín potrebných na vykonanie celého protokolu ako súčet trvaní všetkých krokov.
- **(0,1b) get_hours_for_expert(expert)** – metóda vráti celkový počet hodín, počas ktorých sa daný expert podieľa na realizácii protokolu. Parameter `expert` je objekt

niektorej triedy odvodených od `Expert`. Pri výpočte berte do úvahy iba tie kroky, na ktorých sa môže expert podieľať podľa požadovaného typu experta.

Trieda `Experiment` – 2 body

Trieda `Experiment` reprezentuje konkrétny experiment, v ktorom sa testuje jeden AI systém podľa definovaného hodnotiaceho protokolu za účasti vybraných expertov. Experiment simuluje postupné vykonávanie jednotlivých krokov protokolu počas viacerých dní a po dokončení poskytuje výsledné hodnotenie systému. Trieda obsahuje nasledovné členské premenné:
- `lab` – `ResearchLab` – laboratórium, ktoré experiment realizuje,
- `protocol` – `EvaluationProtocol` – protokol určujúci jednotlivé kroky experimentu,
- `ai_system` – `AISystem` – AI systém, ktorý je predmetom experimentu,
- `experts` – `list` – zoznam expertov zapojených do experimentu,
- `remaining_hours_per_step` – `dict` – slovník uchovávajúci zostávajúci počet hodín pre jednotlivé kroky protokolu.

Implementujte konštruktor (0,5b), ktorý nastaví hodnoty členských premenných podľa vstupných parametrov. Po vytvorení objektu inicializujte slovník `remaining_hours_per_step` tak, že pre každý krok protokolu nastavíte počet zostávajúcich hodín potrebných na jeho dokončenie. Pre každý krok protokolu zistite, ktorý expert sa môže na danom kroku podieľať, pre jednoduchosť môžete rátať s tým, že stále bude iba jeden takýto expert. Počet hodín upravte na počet skutočne potrebných hodín vzhľadom na schopnosti experta (pomocná metóda v triede `Expert`), pričom základný čas získate z objektu `Step`. Výsledné hodnoty uložte do slovníka `remaining_hours_per_step`, kde kľúčom je objekt typu `Step` a hodnotou počet zostávajúcich hodín.

V triede potrebujete implementovať ďalšie metódy:
- **(0,4b) simulate_day()** – metóda nasimuluje jeden deň experimentu. Každý expert môže za jeden deň odpracovať maximálne 6 hodín na experimente. Pre každý krok protokolu znížte počet zostávajúcich hodín o 6 za každého experta, ktorý sa môže na danom kroku podieľať. Zabezpečte, aby počet zostávajúcich hodín nikdy neklesol pod 0.
- **(0,2b) is_finished()** – metóda vracia hodnotu `True`, ak boli dokončené všetky kroky protokolu (pre každý krok je počet zostávajúcich hodín rovný 0). V opačnom prípade vracia hodnotu `False`.
- **(0,6b) compute_results()** – metóda vypočíta výsledky experimentu po jeho dokončení. Výpočet prebieha v niekoľkých krokoch:
    1. získajte jazykové hodnotenie pomocou experta typu `LinguistExpert` a metódy `evaluate_dialogue`
    2. získajte výsledky technickej analýzy pomocou experta typu `MLResearcher` a metódy `validate_model()`. Počet hodín potrebných na analýzu získajte zo všetkých krokov vyžadujúcich ML výskumníka
    3. výsledné hodnotenie vypočítajte pomocou experta typu `Statistician` a metódy `aggregate_scores()`
    4. Metóda vracia slovník obsahujúci všetky čiastkové aj finálne výsledky experimentu s nasledovnou štruktúrou:

    {
    "*quality*": (ai_system, quality_score),
    "*linguist*": (linguist_expert, linguist_score),

"*MLexpert*": (ml_expert, (coherences, consistencies, variability)),
"*Statistician*": (statistician, aggregate_scores)
},
kde hodnota je dvojica, kde prvý prvok je AI systém (pre *quality*), resp. expert vykonávajúci hodnotenie (pre ostatné kľúče), a druhý prvok je výsledné skóre (základná kvalita AI systému pre *quality* a výsledky vyhodnotenia jednotlivými expertmi pre ostatné kľúče).

- **(0,1b) employs(expert)** – metóda vracia hodnotu `True`, ak sa daný expert zúčastňuje experimentu, inak vracia `False`.
- **(0,2b) tests(ai_system, protocol)** – metóda vracia hodnotu `True`, ak experiment testuje zadaný AI systém podľa zadaného protokolu, v opačnom prípade vracia `False`. Metóda slúži ako pomocná metóda pre triedu `ResearchLab`.

Trieda `ResearchLab` – 1 bod

Trieda `ResearchLab` reprezentuje výskumné laboratórium, ktoré organizuje experimenty hodnotiace AI systémy. Laboratórium má k dispozícii expertov rôznych špecializácií a zoznam požiadaviek na experimenty. Počas simulácie vyberá vhodné experimenty podľa zvolenej stratégie a postupne ich realizuje až do ich ukončenia. Trieda obsahuje nasledovné členské premenné:
- `members` – `list` – zoznam expertov pracujúcich v laboratóriu (objekty tried odvodených od `Expert`),
- `requests` – `list` – zoznam požiadaviek na experimenty; každá požiadavka je dvojica `(ai_system, protocol)`,
- `active_experiments` – `list` – zoznam práve prebiehajúcich experimentov (objekty triedy `Experiment`),
- `completed_experiments` – `list` – zoznam dokončených experimentov,
- `selection_strategy` – `ExperimentStrategy` – stratégia určujúca výber experimentov.

Konštruktor triedy nastaví zoznam expertov a zoznam požiadaviek na experimenty. Zoznam aktívnych a dokončených experimentov inicializuje ako prázdne zoznamy. Stratégiu výberu experimentov nastaví na hodnotu `None`. Konštruktor už nemusíte upravovať.

V triede implementujte nasledovné metódy:

- `set_strategy(strategy)` – metóda nastaví stratégiu výberu experimentov. Parameter `strategy` je objekt triedy odvodených od `ExperimentStrategy`. Stratégiu uložte do členskej premennej `selection_strategy`. Metóda je už plne implementovaná.
- **(0,2b) get_available_experts(expert_type)** – metóda vráti zoznam expertov, ktorí sú momentálne dostupní pre experiment. Expert je dostupný, ak sa momentálne nepodieľa na žiadnom aktívnom experimente. Parameter `expert_type` môže obsahovať konkrétnu triedu experta (napr. `LinguistExpert`). Ak je parameter zadaný, metóda vracia iba expertov daného typu. Ak parameter nie je zadaný, metóda vracia všetkých dostupných expertov.
- **(0,2b) get_unsolved_requests()** – metóda vráti zoznam požiadaviek na experimenty, ktoré ešte neboli realizované. Požiadavka sa považuje za vyriešenú, ak už existuje experiment, ktorý testuje rovnaký AI systém podľa rovnakého protokolu.

- **(0,2b) can_run_protocol(protocol)** – metóda zistí, či má laboratórium dostatok dostupných expertov na realizáciu zadaného protokolu (parameter `protocol` typu `EvaluationProtocol`). Pre každý požadovaný typ experta musí existovať aspoň jeden dostupný expert daného typu. Metóda vracia hodnotu `True`, ak je možné protokol realizovať, inak vracia hodnotu `False`.
- **(0,4b) run_simulation()** – metóda spustí simuláciu experimentov. Pred spustením simulácie skontrolujte, či bola nastavená stratégia výberu experimentov. Ak stratégia nebola nastavená, vyvolajte výnimku typu `RuntimeError` so správou *No selection strategy was set*.
  Simulácia prebieha po jednotlivých dňoch, kým nie sú spracované všetky požiadavky. Každý deň vykonajte nasledovné kroky:
  1. pomocou stratégie výberu experimentov vytvárajte nové experimenty, pokiaľ je to možné,
  2. nové experimenty pridajte do zoznamu aktívnych experimentov a odstráňte príslušné požiadavky zo zoznamu `requests`,
  3. pre každý aktívny experiment zavolajte metódu `simulate_day()`, aby ste aktualizovali počet zostávajúcich hodín,
  4. skontrolujte, ktoré experimenty boli dokončené, a presuňte ich zo zoznamu aktívnych experimentov do zoznamu dokončených experimentov,
  5. zvýšte počet dní simulácie o 1.

  Metóda vracia celkový počet dní potrebných na vykonanie všetkých experimentov ako celé číslo (do úvahy berte počet začatých dní).

Trieda `ExperimentStrategy` – 0,5 bodu

Trieda `ExperimentStrategy` reprezentuje všeobecnú stratégiu výberu experimentov, ktoré bude výskumné laboratórium realizovať. Ďalšie logiky výberu experimentu budú implementované v podtriedach, ktoré budú reprezentovať rôzne rozhodovacie prístupy. Trieda obsahuje iba jednu členskú premennú, `lab` typu `ResearchLab`, teda referenciu na laboratórium, ktoré stratégiu používa. Konštruktor inicializuje členskú premennú `lab` na hodnotu `None`. Tá je nastavená až v metóde `assign_to_lab(lab)`, ktorú máte plne implementovanú.

V triede implementujte metódu **`create_experiment(requests)` (0,5b)**, ktorá vyberie vhodnú požiadavku na experiment a vytvorí nový objekt triedy `Experiment`. Parameter `requests` predstavuje zoznam požiadaviek na experimenty, pričom každá požiadavka je dvojica `(ai_system, protocol)`. Metóda pracuje nasledovne:

1. ak stratégia nemá nastavené laboratórium, metóda vráti hodnotu `None` (nevygeneruje sa výnimka),
2. zo zoznamu požiadaviek vyberte iba tie, ktoré ešte neboli vyriešené,
3. z vybraných požiadaviek určte tie, ktoré je možné realizovať vzhľadom na dostupnosť expertov,
4. ak neexistuje žiadna realizovateľná požiadavka, metóda vracia hodnotu `None`,
5. inak náhodne vyberte jednu požiadavku a priraďte jej náhodne expertov:
   - zistite zoznam potrebných typov expertov
   - a pre každý typ experta vyberte jedného náhodného dostupného experta,
6. vytvorte nový objekt triedy `Experiment` s vybraným AI systémom, protokolom a zoznamom expertov,
7. metóda vracia vytvorený objekt triedy `Experiment`.

Ďalšie výbery implementujte v podtriedach s nasledovnou logikou:

- (0,5b) `ChronologicalStrategy` – implementuje stratégiu výberu experimentov, ktorá uprednostňuje požiadavky podľa poradia, v akom boli zadané (pozícia v zozname `requests`). Zo zoznamu realizovateľných požiadaviek vyberte prvú požiadavku v poradí. Pri výbere neberte do úvahy žiadne ďalšie kritériá.
- (0,5b) `ComplexProtocolStrategy` – implementuje stratégiu, ktorá uprednostňuje experimenty s najkomplexnejším hodnotiacim protokolom. Zo zoznamu realizovateľných požiadaviek vyberte tú, ktorej protokol má najväčší celkový počet hodín. Ak existuje viac požiadaviek s rovnakým celkovým trvaním, môžete vybrať ľubovoľnú z nich.
- (0,5b) `EfficientStrategy` – implementuje stratégiu, ktorá tiež uprednostňuje experimenty s najdlhším protokolom. Zo zoznamu realizovateľných požiadaviek vyberte tú, ktorej protokol má najväčší celkový počet hodín. Projektu stále priraďte najviac skúseného dostupného experta.

V súbore `main.py` máte pripravený ukážkový kód na generovanie a spustenie náhodnej simulácie. Kód môžete ľubovoľne upravovať, slúži len ako pomôcka pre lepšie pochopenie prepojenia jednotlivých tried.